

Wprowadzenie do głębokich architektur sieci neuronowych

Trenowanie sieci neuronowych

Wykład nr 3

dr Mateusz Draga

Zakład Hydrobiologii

matdra@amu.edu.pl

Zagadnienia na tym wykładzie



Wydział
Biologii

- Forward Propagation
- Backpropagation
- Loss function
- Learning rate
- Gradient descent
- Epoki
- Dane testowe
- Dane treningowe

1. Przeprowadź forward propagation

- Dla danych wejściowych oblicz wartości aktywacji kolejnych warstw aż do uzyskania predykcji sieci neuronowej.

2. Oblicz funkcję straty (loss function) i błąd modelu (Error)

- Porównaj przewidziany wynik z rzeczywistą wartością i wyznacz błąd modelu, określający jak bardzo predykcja odbiega od poprawnej odpowiedzi.

3. Rozpocznij backpropagation

- Propaguj błąd od warstwy wyjściowej do pierwszej warstwy sieci.
- Dla każdej warstwy oblicz gradienty funkcji straty względem wag i biasów.

4. Zastosuj regułę łańcuchową

- Sieć neuronową można traktować jako złożenie wielu zagnieżdżonych funkcji. Wykorzystując regułę łańcuchową rachunku różniczkowego obliczamy, jaki wpływ mają poszczególne wagi i biasy na końcowy błąd modelu.

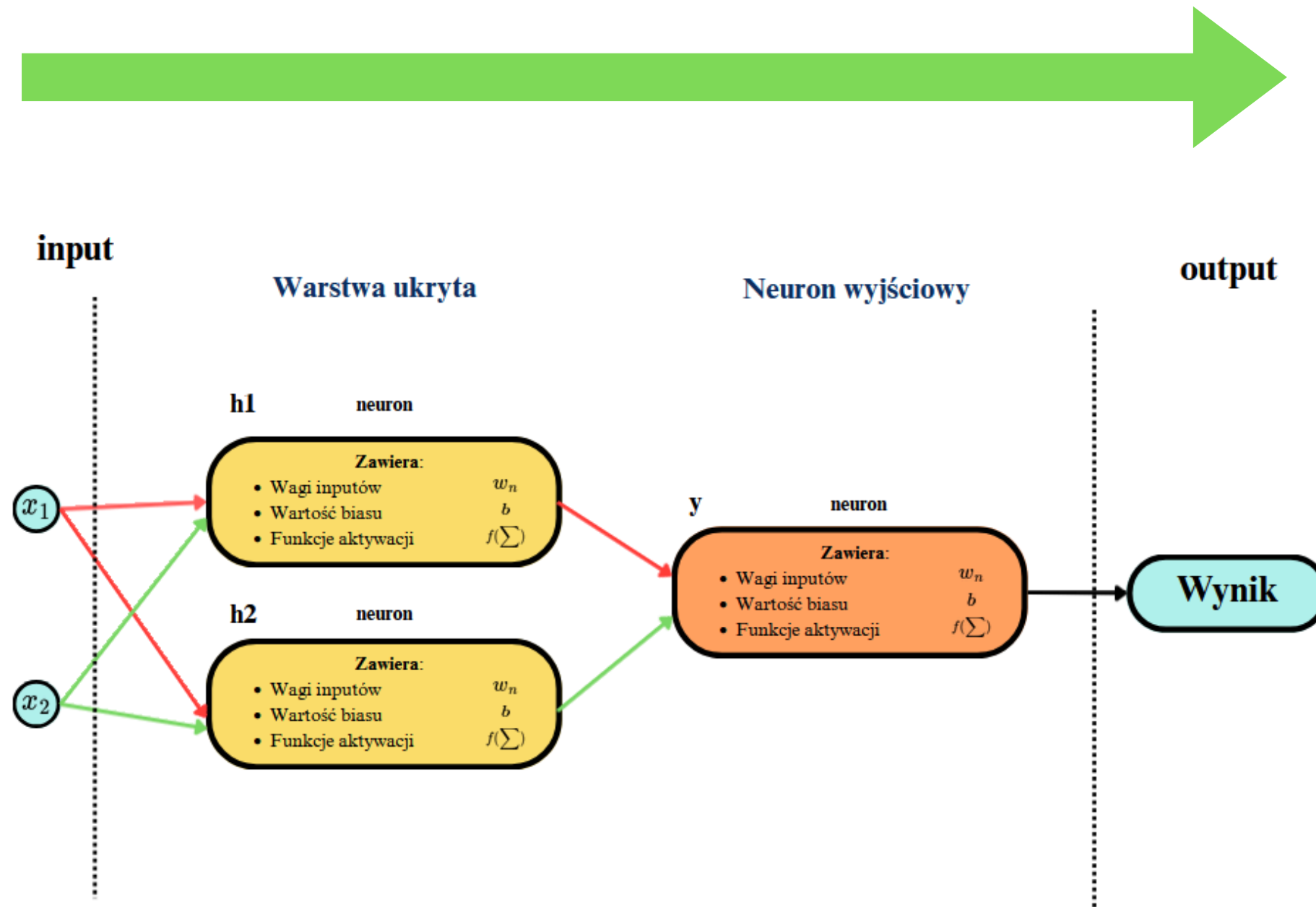
5. Wykonaj aktualizację parametrów (Gradient Descent)

- Na podstawie obliczonych gradientów aktualizujemy wartości wag i biasów w kierunku zmniejszającym wartość funkcji straty.

1) Forward Propagation



Wydział
Biologii



Sieć neuronowa to złożenie funkcji: każda kolejna warstwa aplikuje swoją transformację do wyniku poprzedniej, tworząc głęboko zagnieżdżoną funkcję postaci $f_1(f_2(f_3(\dots(x)\dots)))$.

2) Błąd modelu i funkcja strat

Jak dobrze nam poszło?



Wydział
Biologii

Do liczenia błędu modelu (Error) wykorzystujemy funkcje strat (loss function).

Najważniejsze pytanie to: jak zmienić wagi oraz bias by jak najbardziej zminimalizować “Error”?

W zależności od typu problemu stosujemy różne funkcje strat.

y - faktyczny wynik

$\hat{y}$ - przewidziany wynik

L - strata (“Loss”)

$$\hat{y} = f_{n-1} (\dots (f_2 (f_1 (x))))$$

$$L = f_n (\dots (f_2 (f_1 (x))))$$

$J(\theta)$

Loss function, Cost function



Wydział
Biologii

Regresja

- Mean Squared Error (**MSE**)

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- Mean Absolute Error (**MAE**)

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Klasyfikacja

- Binary Cross-Entropy (**BCE**)

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

- Categorical Cross-Entropy (**CCE**)

$$L = -\sum_{i=1}^C y_i \log(\hat{y}_i)$$

- Focal Loss

$$FL(p_t) = -\alpha(1 - p_t)^\gamma \log(p_t)$$

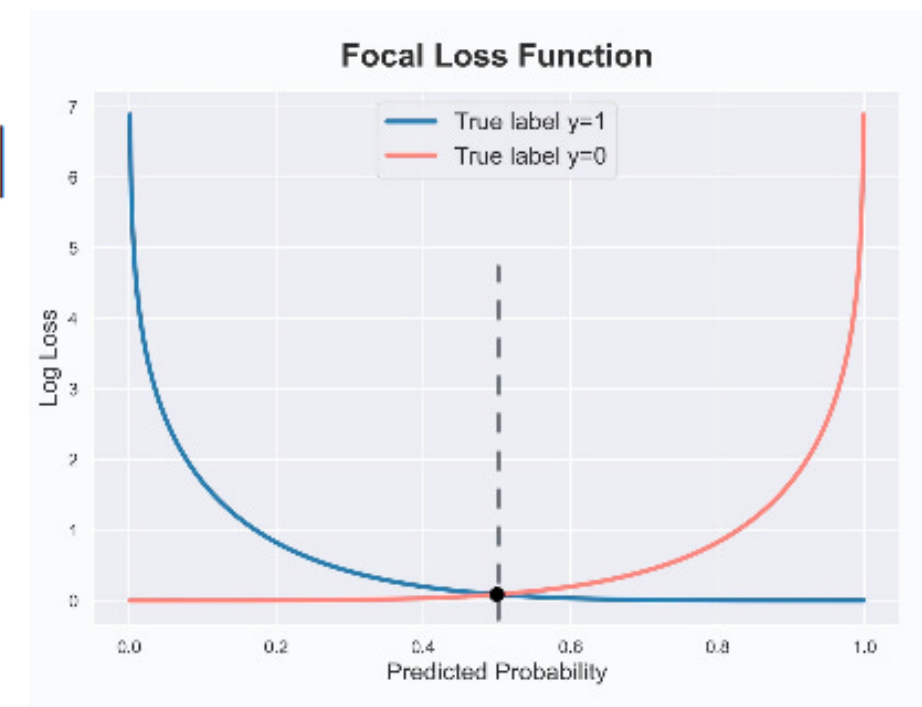
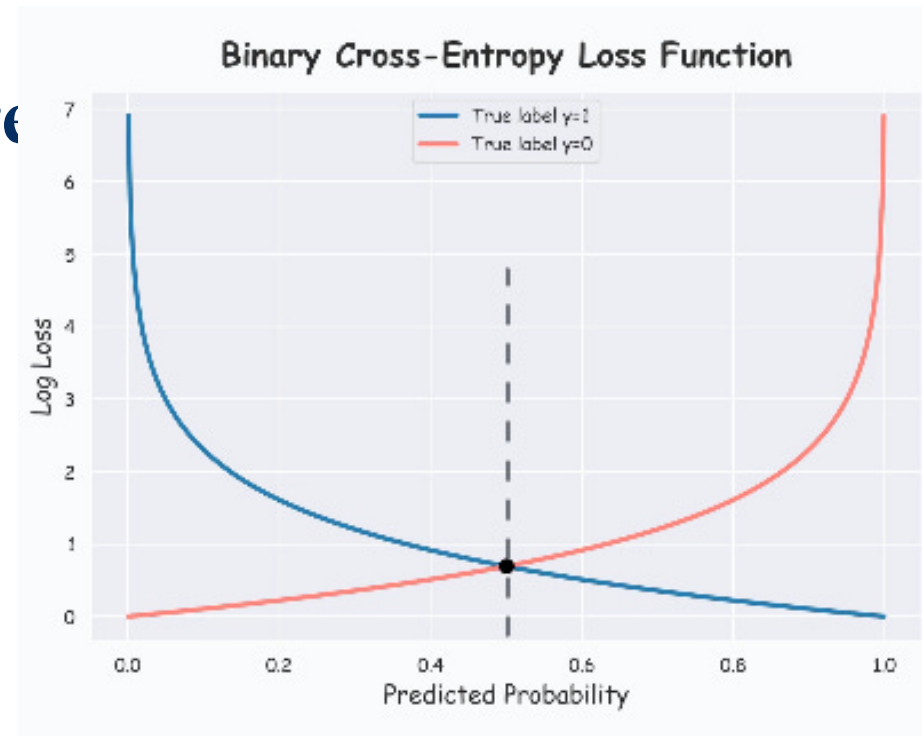
Regr

- Mean Squared Error (MSE)

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- Mean Absolute Error (MAE)

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$



Klasyfikacja

- Binary Cross-Entropy (BCE)

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

- Categorical Cross-Entropy (CCE)

$$L = -\sum_{i=1}^C y_i \log(\hat{y}_i)$$

- Focal Loss

$$FL(p_t) = -\alpha(1 - p_t)^\gamma \log(p_t)$$

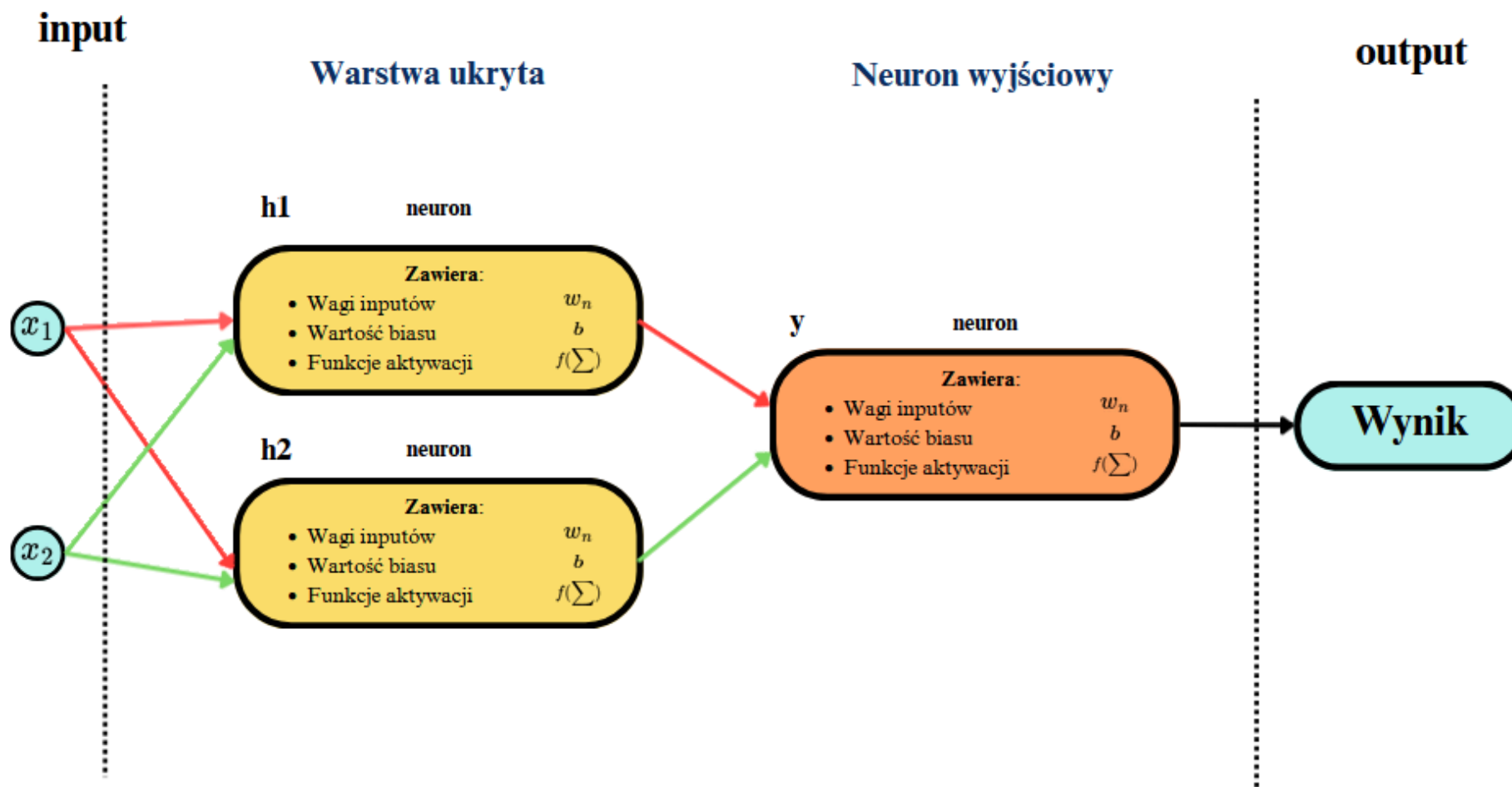
3) Backpropagation



Wydział
Biologii



Poprawiamy od końca wagi i biasy dla obliczonego L.



$$\frac{dL}{dw}$$

$$\frac{\delta L}{\delta w}$$

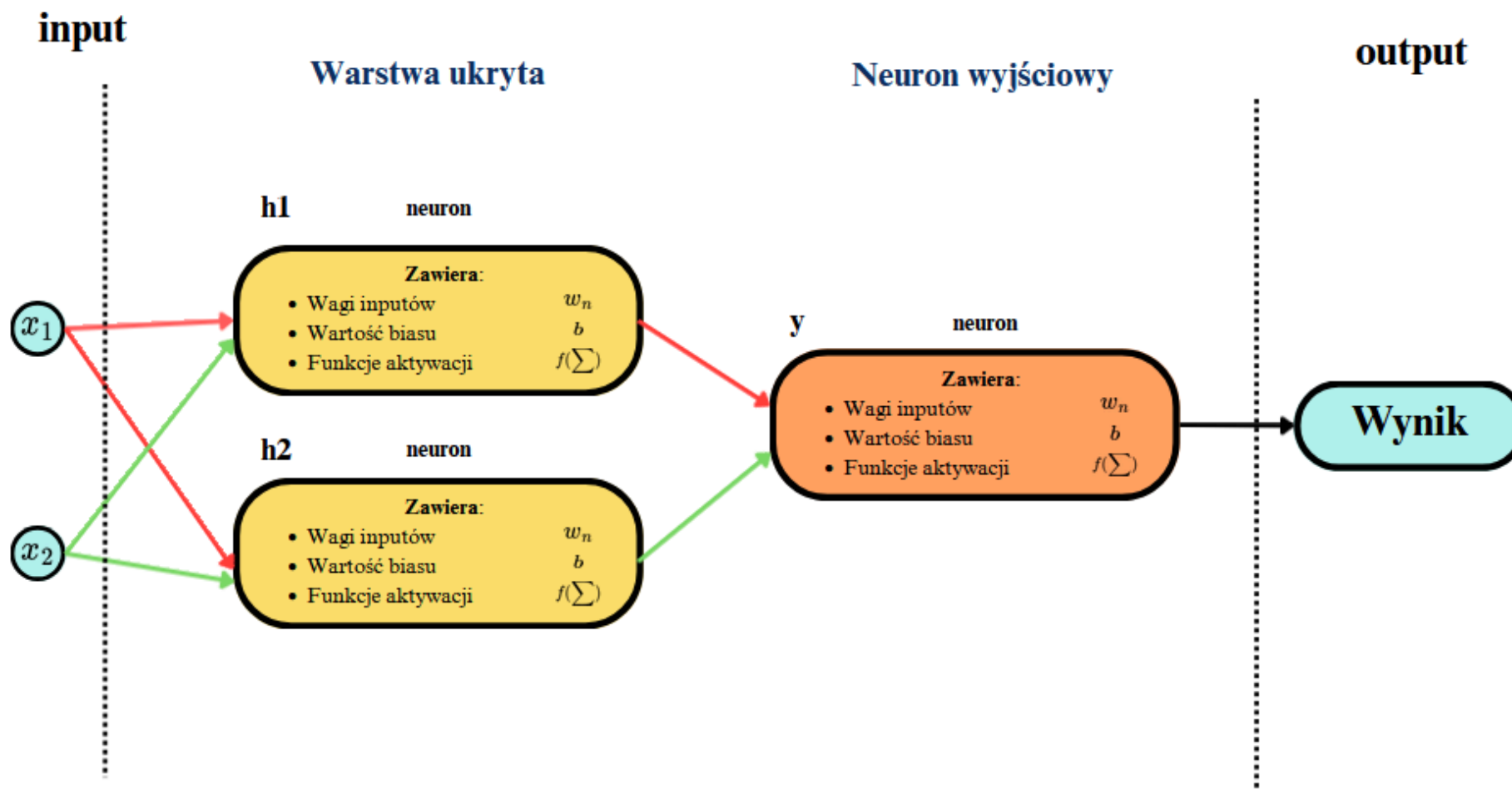
3) Backpropagation



Wydział
Biologii

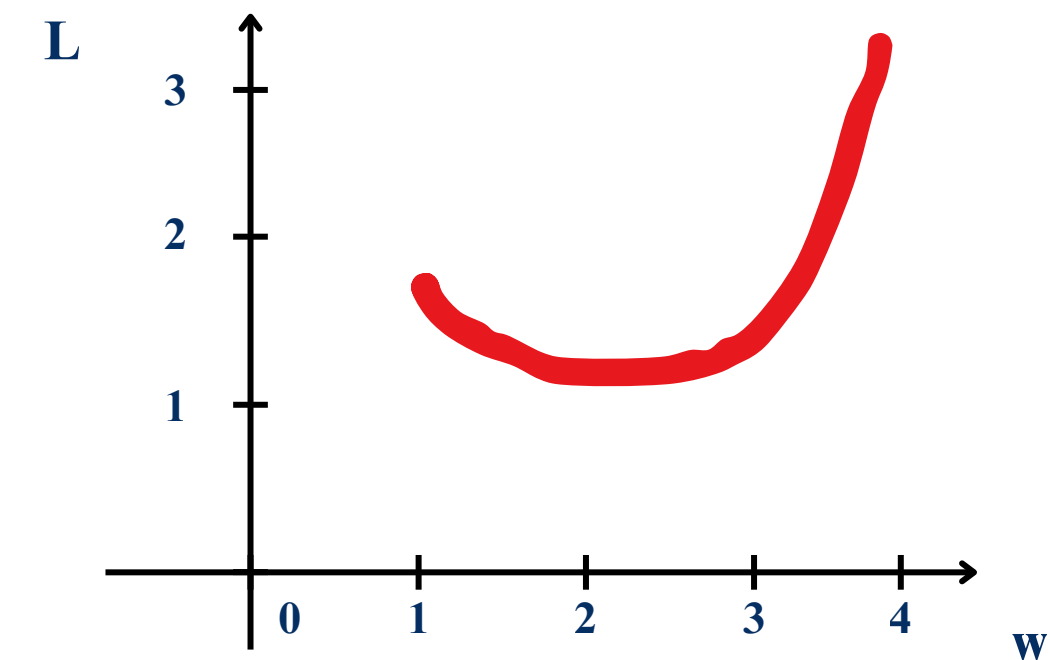


Poprawiamy od końca wagi i biasy dla obliczonego L.



$$\frac{dL}{dw}$$

$$\frac{\delta L}{\delta w}$$



4) Metoda Łańcuchowa



Wydział
Biologii

By wiedzieć, jak należy zmienić wartości wag_i biasów, musimy obliczyć gradient funkcji straty (cost function) względem wszystkich parametrów sieci. Gradient informuje nas:

- w którą stronę należy zmienić parametr,
- oraz jak duży wpływ ma on na końcowy błąd modelu.

Najmocniej aktualizujemy te wagi i biasy, które w największym stopniu przyczyniają się do powstawania błędu.

Jak to określić? Sieć neuronową można traktować jako złożenie wielu zagnieżdżonych funkcji, gdzie wynik jednej warstwy staje się wejściem kolejnej. Ostateczna predykcja zależy więc od wszystkich wcześniejszych operacji wykonywanych w sieci.

Podczas backpropagation obliczamy gradienty, propagując błąd od warstwy wyjściowej do wejściowej. W tym celu wykorzystujemy regułę łańcuchową rachunku różniczkowego, która pozwala określić, jak zmiana pojedynczej wagi lub biasu wpływa na końcową wartość funkcji straty.

4) Metoda Łańcuchowa



Wydział
Biologii

By wiedzieć, jak należy zmienić wartości wag_i biasów, musimy obliczyć gradient funkcji straty (cost function) względem wszystkich parametrów sieci. Gradient informuje nas:

- w którą stronę należy zmienić parametr,
- oraz jak duży wpływ ma on na końcowy błąd modelu.

Najmocniej aktualizujemy te wagi i biasy, które w największym stopniu przyczyniają się do powstawania błędu.

Jak to określić? Sieć neuronową można traktować jako złożenie wielu zagnieżdżonych funkcji, gdzie wynik jednej warstwy staje się wejściem kolejnej. Ostateczna predykcja zależy więc od wszystkich wcześniejszych operacji wykonywanych w sieci.

Podczas backpropagation obliczamy gradienty, propagując błąd od warstwy wyjściowej do wejściowej. W tym celu wykorzystujemy regułę łańcuchową rachunku różniczkowego, która pozwala określić, jak zmiana pojedynczej wagi lub biasu wpływa na końcową wartość funkcji straty.

Dla trywialnego neuronu bez biasu oraz funkcji aktywacji uproszczony rachunek wygląda następująco:

$$\hat{y} = wx$$

$$w \rightarrow \hat{y} \rightarrow L$$

$$\frac{dL}{dw} = \frac{dL}{d\hat{y}} \times \frac{d\hat{y}}{dw}$$

5) Gradient Descent



Wydział
Biologii

wzór na aktualizację wag

$$w = w - \eta \frac{\delta L}{\delta w}$$

η - learning rate

Gradient Descent to algorytm optymalizacji używany do minimalizacji funkcji straty poprzez stopniową aktualizację wag i biasów sieci neuronowej. W każdej iteracji parametry są zmieniane w kierunku przeciwnym do gradientu, ponieważ gradient wskazuje kierunek najszybszego wzrostu błędu.

Gradient pomaga znaleźć takie parametry, które minimalizują błąd.

Szukamy takiego θ dla którego funkcja kosztu $J(\theta)$ jest minimalna.

Musimy dowiedzieć się w “którą stronę funkcja rośnie najszybciej” i iterując iść w przeciwnym kierunku.

$$\theta = \theta - \eta \nabla J(\theta)$$

5) Gradient Descent



Wydział
Biologii

wzór na aktualizację wag

$$w = w - \eta \frac{\delta L}{\delta w}$$

η - learning rate

$$\theta = \theta - \eta \nabla J(\theta)$$

Rodzaje:

Batch Gradient Descent

- Gradient obliczany jest na podstawie całego zbioru treningowego przed wykonaniem aktualizacji wag. Metoda jest stabilna i dokładna, ale może być bardzo wolna dla dużych datasetów.

Stochastic Gradient Descent (SGD)

- Gradient obliczany jest dla pojedynczej próbki treningowej, a wagi aktualizowane są po każdym przykładzie. Metoda działa szybciej i zużywa mniej pamięci, ale aktualizacje są bardziej chaotyczne i mniej stabilne.

Mini-Batch Gradient Descent

- Najczęściej stosowana wersja w deep learningu. Gradient liczony jest dla małych batchy danych (np. 32 lub 128 próbek), co stanowi kompromis między szybkością SGD a stabilnością Batch Gradient Descent.

Batch to niewielka grupa próbek treningowych przetwarzanych jednocześnie przez sieć neuronową podczas jednego kroku uczenia.

Dla całego batcha:

1. wykonywany jest forward propagation,
2. obliczany jest loss dla każdej próbki,
3. lossy są agregowane (najczęściej średnia),
4. wykonywany jest jeden backpropagation,
5. następuje jedna aktualizacja wag i biasów.

Przykład:

- dataset: 1000 próbek,
- batch size: 100,

oznacza to, że sieć jednocześnie przetwarza 100 przykładów i aktualizuje wagi dopiero po obliczeniu średniego błędu dla całego batch'a. W jednej epoce wykorzysta się zatem dane podzielone na 10 batch'y

Najczęściej batch'e są losowo tasowane (shuffle) przed każdą epoką.

Epoch (Epoka) oznacza jedno pełne przejście przez cały zbiór treningowy.

Przykład:

dataset: 1000 próbek,
batch size: 100,
w jednej epoce model wykona 10 iteracji.

Jeśli mamy 1000 próbek, batch size równy 100 oraz 10 epok, to sieć wykona łącznie 100 iteracji treningowych (steps). Każda próbka danych zostanie wykorzystana około 10 razy - po jednym razie w każdej epoce.

Przykład:



Wydział
Biologii

Podział danych w procesie uczenia

training, validation & test data



Wydział
Biologii

training data - Dane treningowe służą do uczenia modelu sieci neuronowej. Na ich podstawie model aktualizuje swoje wagi oraz biasy i uczy się rozpoznawania wzorców w danych.

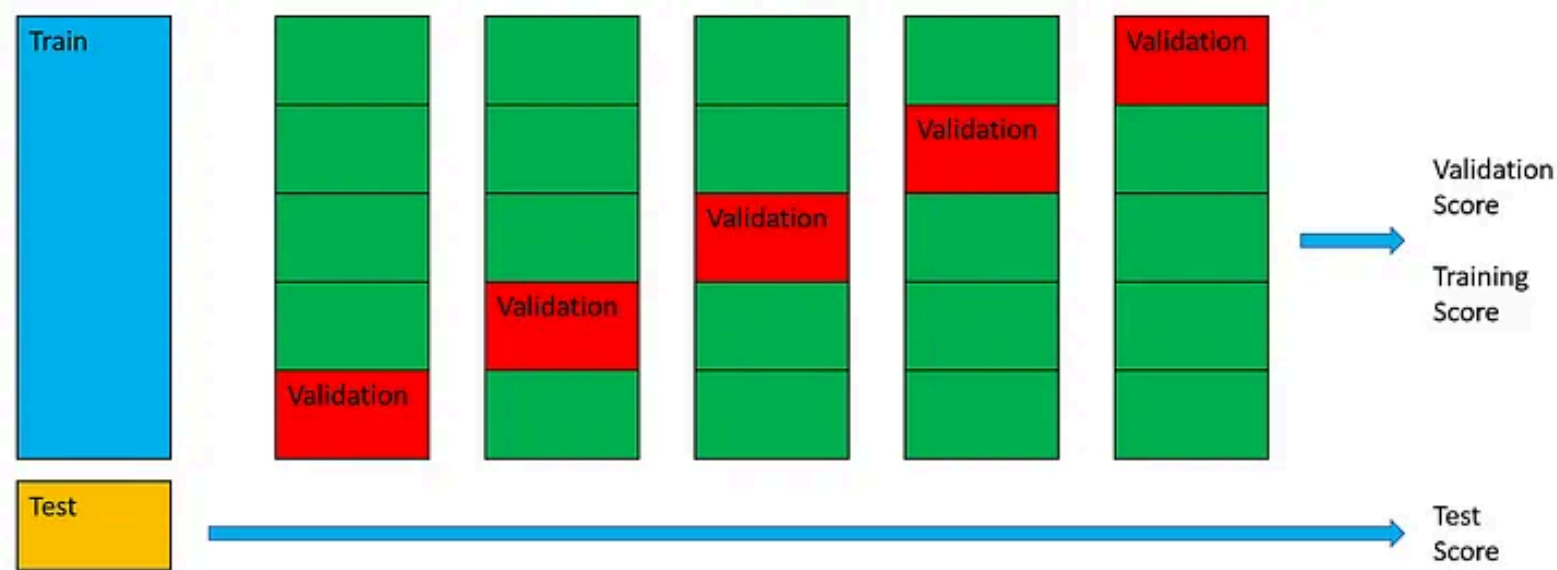
validation data - Dane walidacyjne są wykorzystywane podczas treningu do oceny jakości modelu i dostrajania hiperparametrów. Model nie uczy się bezpośrednio na tych danych, dzięki czemu można sprawdzić, czy nie dochodzi do przeuczenia (overfittingu).



za Jainvidip, <https://medium.com/@jainvidip/understanding-train-test-and-validation-data-in-machine-learning-f8276165619c>, data dostępu 20.05.2026

test data - Dane testowe służą do końcowej oceny skuteczności po wytrenowaniu modelu. Są używane dopiero po zakończeniu treningu i pozwalają ocenić, jak model poradzi sobie z nowymi, wcześniej niewidzianymi danymi.

Cross-Validation



Cross-validation - to technika oceny modelu polegająca na wielokrotnym trenowaniu i testowaniu modelu na różnych podziałach danych. Najczęściej stosuje się metodę k-fold, w której zbiór danych dzieli się na k części - model trenuje się na $k-1$ częściach, a testuje na pozostałej. Proces powtarza się tyle razy, aby każda część została użyta jako zbiór testowy. Dzięki temu można dokładniej ocenić skuteczność modelu i zmniejszyć ryzyko przypadkowego dopasowania do jednego podziału danych.

za Jainvidip, <https://medium.com/@jainvidip/understanding-train-test-and-validation-data-in-machine-learning-f8276165619c>, data dostępu 20.05.2026

1. **Wczytaj dane**
2. **Wybierz cechy wejściowe (X) oraz wartość do przewidywania (y)**
3. **Przygotuj dane** (standaryzacja, one-hot encoding...)
4. **Podziel dane** na zbiory treningowe, walidacyjne i testowe
5. **Zamień dane** na tensory PyTorch
6. **Zdefiniuj architekturę** sieci neuronowej (warstwy, aktywacje)
7. **Wybierz hiperparametry** (funkcję straty i optymalizator)
8. **Uruchom pętlę treningową:**
 - wykonaj predykcję (forward pass),
 - oblicz błąd (loss),
 - policz gradienty (backpropagation),
 - zaktualizuj wagi modelu
9. **Sprawdź wynik na danych walidacyjnych** po każdej epoce
10. **Zakończ trening modelu**
11. **Oceń jakość modelu** na danych testowych
12. **Zapisz model**



DataFrame vs NumPy vs Tensor



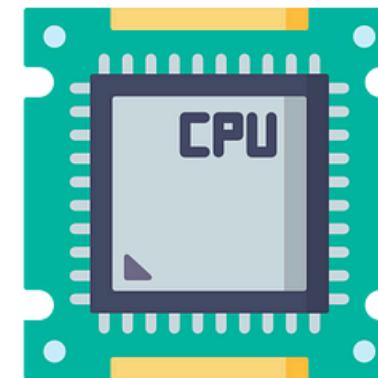
Wydział
Biologii

DataFrame (Pandas) - struktura danych przypominająca tabelę (jak Excel), z nazwanymi kolumnami i indeksami. Jest bardzo wygodna do analizy, czyszczenia i wstępnego przetwarzania danych.

NumPy array - szybka, numeryczna struktura danych przechowująca dane w postaci macierzy lub wektorów. Jest bardziej „surowa” niż DataFrame. Nie ma nazw kolumn ani indeksów, ale jest bardzo wydajna obliczeniowo. Działa na CPU.

Tensor (PyTorch) - Tensor w PyTorch to struktura danych podobna do NumPy array, ale dodatkowo obsługuje GPU i automatyczne liczenie gradientów (autograd). Jest podstawowym formatem danych używanym do trenowania sieci neuronowych.

Arrays



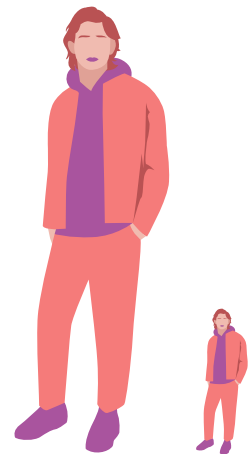
`np.array`

vs.

Tensors



`torch.tensor`



Skalowanie danych (feature scaling) - Skalowanie danych polega na przekształceniu wartości cech do podobnego zakresu, np. o średniej 0 i odchyleniu standardowym 1. Dzięki temu sieć neuronowa uczy się stabilniej i szybciej, ponieważ cechy o dużych wartościach nie dominują nad pozostałymi.

One-Hot Encoding - One-Hot Encoding to metoda zamiany danych kategorycznych na postać numeryczną zrozumiałą dla modelu. Każda kategoria otrzymuje osobną kolumnę zawierającą wartości 0 lub 1, np. „Biolog” $\rightarrow$ [1,0,0], „Informatyk” $\rightarrow$ [0,1,0], „Bioinformatyk” $\rightarrow$ [0,0,1]. Z jednej kolumny danych otrzymujemy więc wiele.



Data Leakage - Data leakage występuje wtedy, gdy model podczas treningu uzyskuje informacje, które nie powinny być dostępne. Przykładem jest dopasowanie skalera do całego zbioru danych przed podziałem na train i test, co powoduje „przeciek” informacji z danych testowych do treningu.

Train loss vs Validation loss i Accuracy



Wydział
Biologii

train loss - mierzy jak dobrze radzi sobie model na train data. Przedstawia więc błąd modelu na danych treningowych i bezpośrednio wpływa na aktualizację wag podczas uczenia. Niższa wartość oznacza, że model dobrze się uczy, natomiast wysoka wartość często wskazuje na niedouczenie (underfitting) lub trudności w wykrywaniu wzorców w danych.

validation loss - ocenia model na danych niewykorzystywanych do uczenia i pomaga wykrywać przeuczenie oraz dobrać hiperparametry. Jeśli train loss jest niski a validation loss jest wysoki, oznacza to, że prawdopodobnie doszło do przeuczenia (overfitting) na danych treningowych.

Train loss vs Validation loss i Accuracy

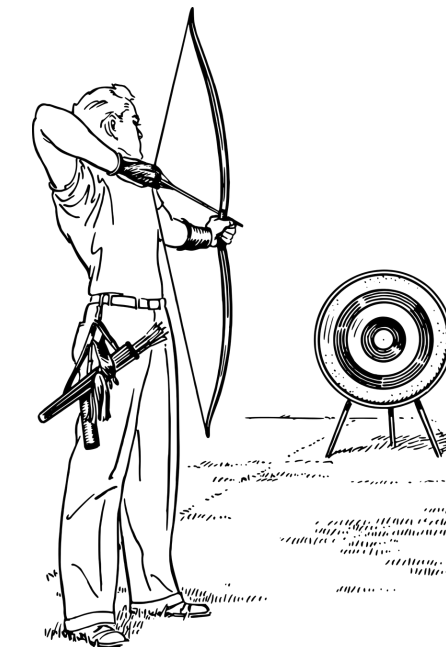


Wydział
Biologii

train loss - mierzy jak dobrze radzi sobie model na train data. Przedstawia więc błąd modelu na danych treningowych i bezpośrednio wpływa na aktualizację wag podczas uczenia. Niższa wartość oznacza, że model dobrze się uczy, natomiast wysoka wartość często wskazuje na niedouczenie (underfitting) lub trudności w wykrywaniu wzorców w danych.

validation loss - ocenia model na danych niewykorzystywanych do uczenia i pomaga wykrywać przeuczenie oraz dobierać hiperparametry. Jeśli train loss jest niski a validation loss jest wysoki, oznacza to, że prawdopodobnie doszło do przeuczenia (overfitting) na danych treningowych.

Accuracy - miara jakości modelu, która określa, jaki procent wszystkich predykcji został poprawnie sklasyfikowany. Oblicza się ją jako stosunek liczby poprawnych przewidywań do całkowitej liczby przykładów. Jest często używana w zadaniach klasyfikacji.



Przykład:



Wydział
Biologii